I/O Memory

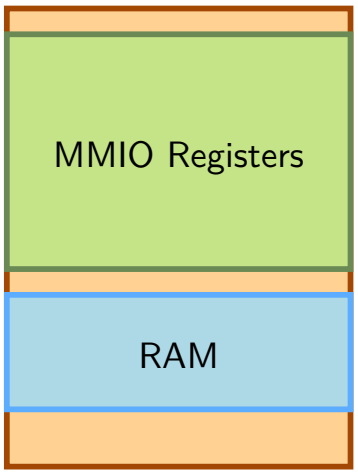


© Copyright 2004-2026, Bootlin
Creative Commons BY-SA 3.0
Corrections, suggestions, contributions and translations are welcome!



Memory-Mapped I/O

- Same address bus to address memory and I/O device registers
- Access to the I/O device registers using regular instructions
- Most widely used I/O method across the different architectures supported by Linux



Physical Memory address space, accessed with normal load/store instructions



Requesting I/O memory

- Tells the kernel which driver is using which I/O registers
- ```
  struct resource *request_mem_region(unsigned long start,
                                      unsigned long len, char *name);
  ```
- ```
  void release_mem_region(unsigned long start, unsigned long len);
  ```
- Allows to prevent other drivers from requesting the same I/O registers, but is purely voluntary.



/proc/iomem example - ARM 32 bit (BeagleBone Black, Linux 5.11)

```
40300000-4030ffff : 40300000.sram sram@0
44e00c00-44e00cff : 44e00c00.prm prm@c00
44e00d00-44e00dff : 44e00d00.prm prm@d00
44e00e00-44e00eff : 44e00e00.prm prm@e00
44e00f00-44e00fff : 44e00f00.prm prm@f00
44e01000-44e010ff : 44e01000.prm prm@1000
44e01100-44e011ff : 44e01100.prm prm@1100
44e01200-44e012ff : 44e01200.prm prm@1200
44e07000-44e07fff : 44e07000.gpio gpio@0
44e09000-44e0901f : serial
44e0b000-44e0bfff : 44e0b000.i2c i2c@0
44e10800-44e10a37 : pinctrl-single
44e10f90-44e10fcf : 44e10f90.dma-router dma-router@f90
48024000-48024fff : 48024000.serial serial@0
48042000-480423ff : 48042000.timer timer@0
48044000-480443ff : 48044000.timer timer@0
```

```
48046000-480463ff : 48046000.timer timer@0
48048000-480483ff : 48048000.timer timer@0
4804a000-4804a3ff : 4804a000.timer timer@0
4804c000-4804cfff : 4804c000.gpio gpio@0
48060000-48060fff : 48060000.mmc mmc@0
4819c000-4819cfff : 4819c000.i2c i2c@0
481a8000-481a8fff : 481a8000.serial serial@0
481ac000-481acfff : 481ac000.gpio gpio@0
481ae000-481aefff : 481ae000.gpio gpio@0
481d8000-481d8fff : 481d8000.mmc mmc@0
49000000-4900ffff : 49000000.dma edma3_cc
4a100000-4a1007ff : 4a100000.ethernet ethernet@0
4a101200-4a1012ff : 4a100000.ethernet ethernet@0
80000000-9fdfffff : System RAM
80008000-80cfffff : Kernel code
80e00000-80f3d807 : Kernel data
```



Mapping I/O memory in virtual memory

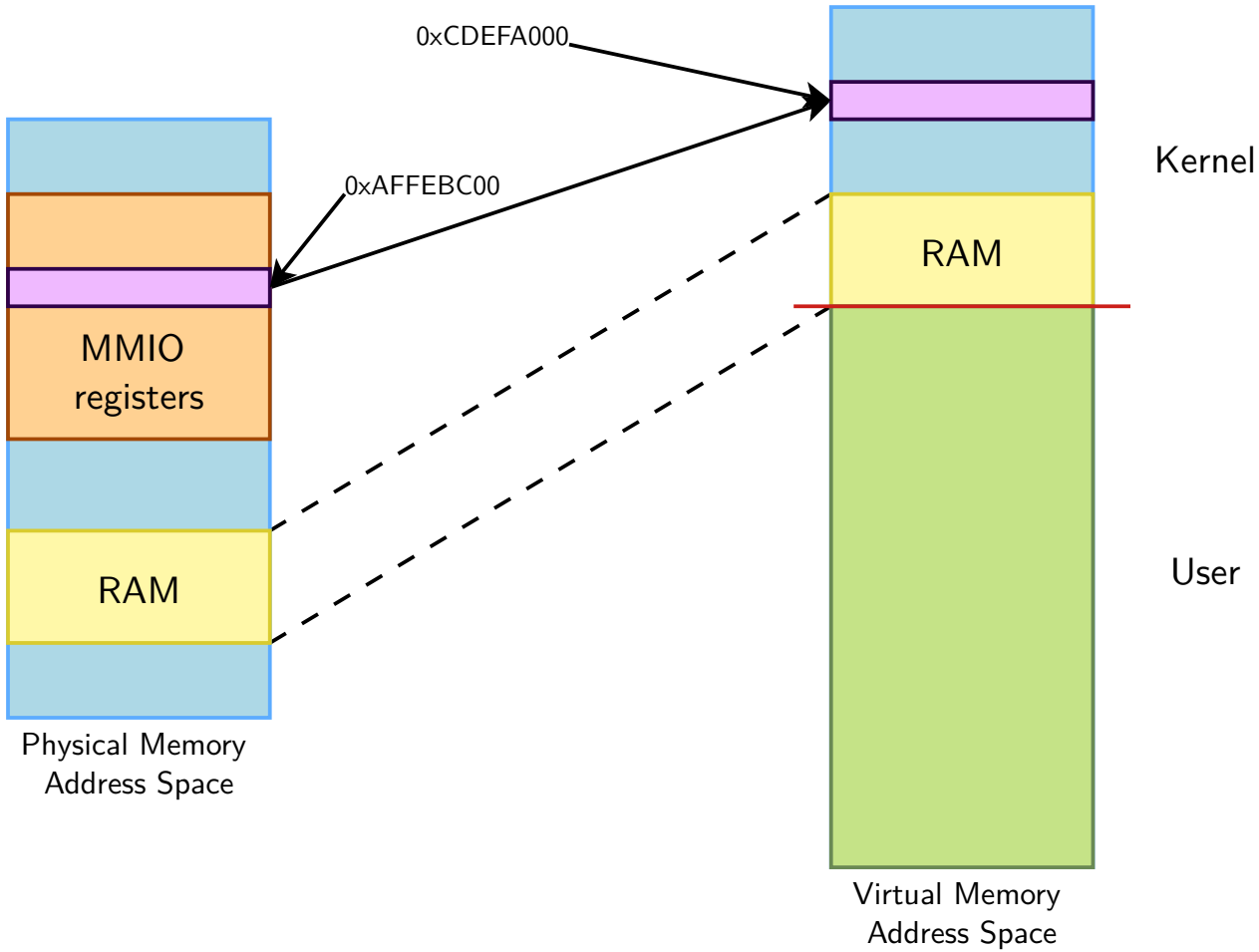
- Load/store instructions work with virtual addresses
- To access I/O memory, drivers need to have a virtual address that the processor can handle, because I/O memory is not mapped by default in virtual memory.
- The `ioremap` function satisfies this need:

```
#include <linux/io.h>

void __iomem *ioremap(phys_addr_t phys_addr, unsigned long size);
void iounmap(void __iomem *addr);
```

- Caution: check that ioremap() doesn't return a `NULL` address!

ioremap()




```
ioremap(0xAFFEBC00, 4096) = 0xCDEFA000
```



Managed API

Instead of calling the following three functions in a row and handle all possible errors:

- platform_get_resource() to retrieve the physical offsets
- request_mem_region() to reserve the region
- ioremap() to create a virtual mapping for the region

it is preferred to use devm_ioremap_resource() instead.

- Caution: unlike the other devm_ functions, its first argument is of type struct platform_device, not a pointer to struct device:

```
base = devm_platform_ioremap_resource(pdev, 0);
if (IS_ERR(base))
        return PTR_ERR(base);
```



Accessing MMIO devices: using accessor functions

- Care must be taken when accessing MMIO registers
  - Memory mapped I/O can be weakly ordered
    - Developer's responsibility to enforcing proper ordering
  - Endianness of the device may be different than the CPU endianness
    - Endianness conversions might be required
  - Directly reading from or writing to addresses returned by ioremap() (*pointer dereferencing*) may not work on some architectures.
- A family of architecture-independent accessor functions are available covering most needs.
  - A few architecture-specific accessor functions also exists.



Ordering

- Reads/writes to MMIO-mapped registers of given device are done in program order
- However reads/writes to RAM can be re-ordered between themselves, and between MMIO-mapped read/writes
- Some of the accessor functions include memory barriers to help with this:
  - Write operation starts with a write memory barrier which prior writes cannot cross
  - Read operation ends with a read memory barrier which guarantees the ordering with regard to the subsequent reads
- Sometimes compiler/CPU reordering is not an issue, in this case the code may be optimized by dropping the memory barriers, using the raw or relaxed helpers
- See Documentation/memory-barriers.txt



MMIO access functions

- `read[b/w/l/q]` and `write[b/w/l/q]` for access to little-endian devices, includes memory barriers
- `ioread[8/16/32/64]` and `iowrite[8/16/32/64]` are very similar to read/write but also work with port I/O (not covered in the course), includes memory barriers
- `ioread[8/16/32/64]be` and `iowrite[8/16/32/64]be` for access to big-endian devices, includes memory barriers
- `__raw_read[b/w/l/q]` and `__raw_write[b/w/l/q]` for raw access: no endianness conversion, no memory barriers
- `read[b/w/l/q]_relaxed` and `write[b/q/l/w]_relaxed` for access to little-endian devices, without memory barriers
- All functions work on a `void __iomem *`



MMIO access functions summary

Name	Device endianness	Memory barriers
`read/write`	little	yes
`ioread/iowrite`	little	yes
`ioreadbe/iowritebe`	big	yes
`__raw_read/__raw_write`	native	no
`read_relaxed/write_relaxed`	little	no

More details at https://docs.kernel.org/driver-api/device-io.html



/dev/mem

- Used to provide user space applications with direct access to physical addresses.
- Usage: open `/dev/mem` and read or write at given offset. What you read or write is the value at the corresponding physical address.
- Used by applications such as the X server to write directly to device memory.
- Easy to use from a shell with the devmem2 program
- For security reasons, on `x86`, `arm`, `arm64`, `riscv`, `powerpc`, `parisc`, `s390`:
  - CONFIG_STRICT_DEVMEM restricts `/dev/mem` to non-RAM addresses (from v5.12)
  - CONFIG_IO_STRICT_DEVMEM goes beyond and only allows to access *idle* I/O ranges (not appearing in `/proc/iomem`).



Clocking and power management



Enabling hardware

In order to become functional, hardware blocks require:

- Power to be applied
- Clocks to be ticking
  - Hardware blocks are connected through hardware buses (AHB, APB, AXI...), their interface logic require an input clock to respond
    - One will experience bus hangs otherwise
  - When a device exposes a bus (eg. a SPI controller), there is typically a second clock whose frequency can be tuned
- Resets to be released



Power handling

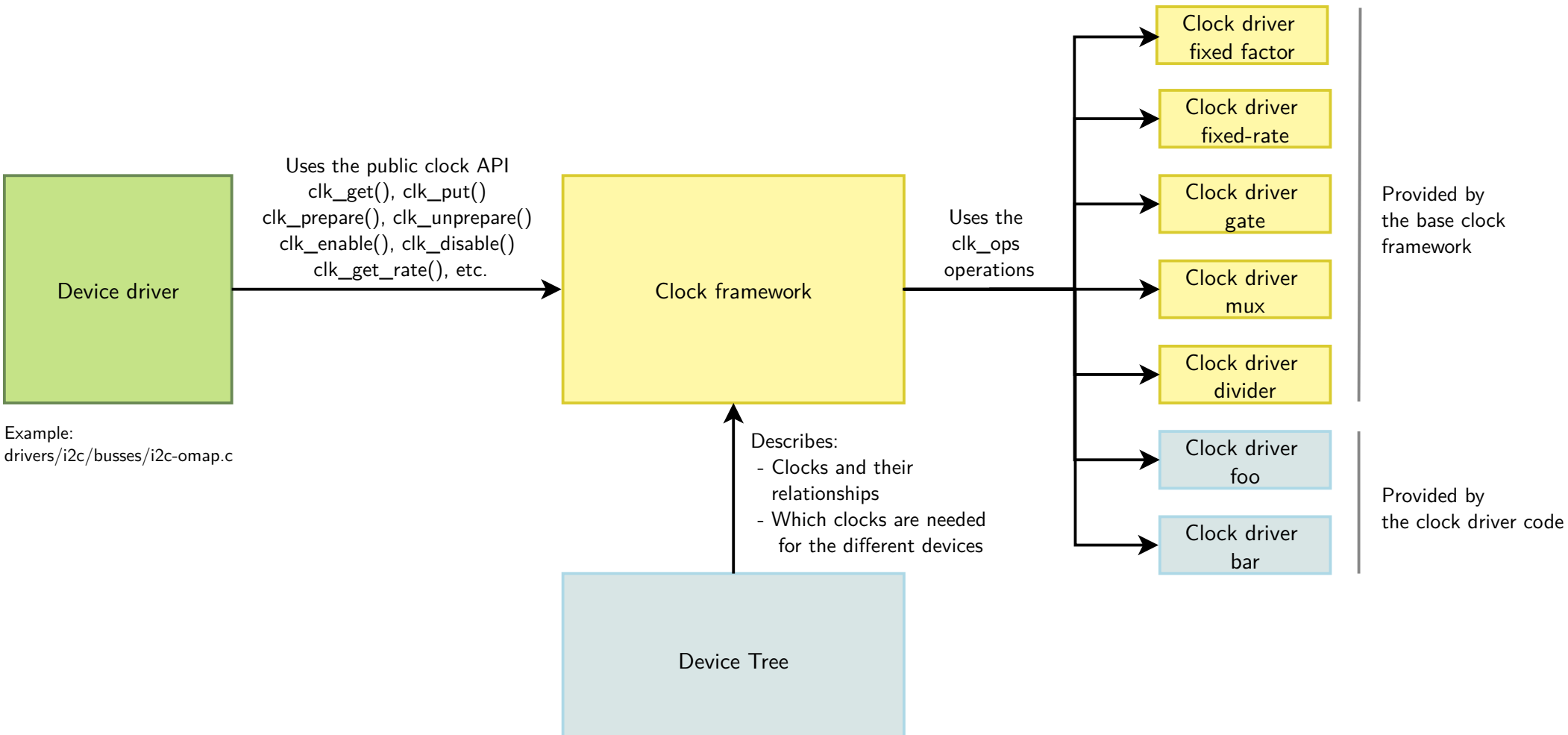
- Internal and external devices can be fed by a regulator
  - Shared refcounted resources
  - See devm_regulator_get_enable()
- Internal devices can typically be part of a power management domain (named `pmdomain`, formerly `genpd`)
  - Shared refcounted resources
  - For example: all display related controllers may be in one power domain (CRTC, LCD controller, HDMI phy, etc). The whole domain is either powered on or off.



Clocks handling

- A clock tree typically starts from the main crystal, feeds PLLs, gates, divisors and reaches every device in the system
  - Shared refcounted resources
  - Managed by the Common Clock Framework (CCF)
  - Simple API described in include/linux/clk.h.
    - devm_clk_get() to lookup and obtain a reference to a clock producer
    - clk_prepare_enable() to inform the system when the clock source should be running
    - clk_disable_unprepare() to inform the system when the clock source is no longer required.
    - clk_get_rate() to obtain the current clock rate (in Hz) for a clock source
    - clk_set_rate() to set the current clock rate (in Hz) of a clock source
- Allows to declare the available clocks and their association to devices in the Device Tree
- Provides a *debugfs* representation of the clock tree
- Is implemented in drivers/clk/

Diagram overview of the common clock framework





Reset handling

- Goal is to put a device in a known harder default state
- Typically done while probing
- Resets can be simple GPIOs
- More complex resets can be registered in the reset control framework
  - Pointed by the `resets` DT property
  - API is straightforward:
    - devm_reset_control_get()
    - reset_control_assert()
    - reset_control_deassert()



Runtime Power Management

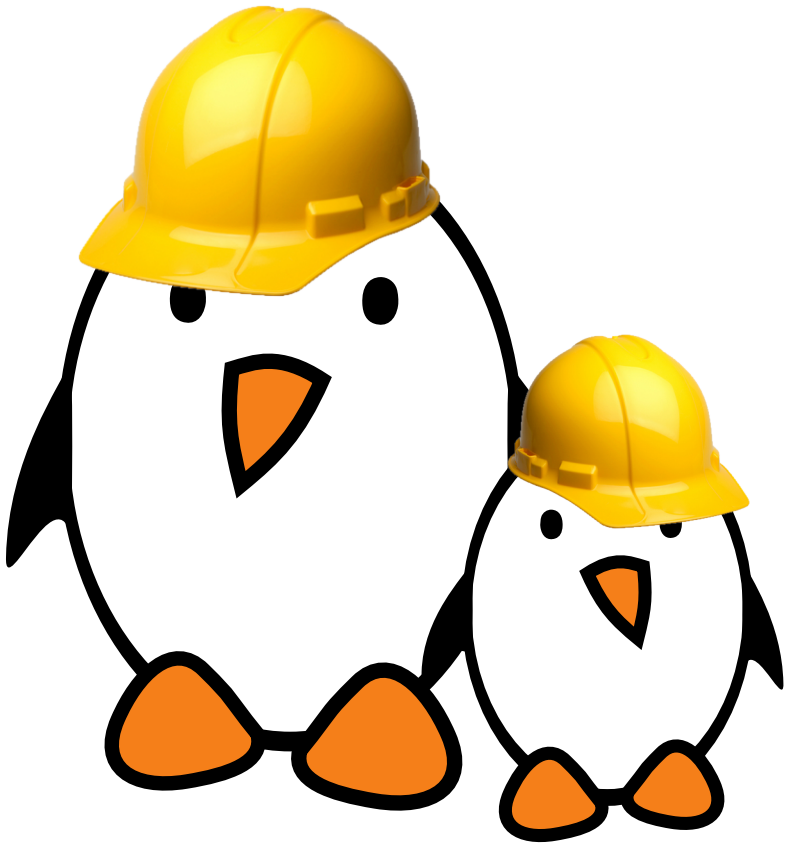
- The state of a device can change at runtime, a sort of per-device idle state
- According to the kernel configuration interface: *Enable functionality allowing I/O devices to be put into energy-saving (low power) states at run time (or autosuspended) after a specified period of inactivity and woken up in response to a hardware-generated wake-up event or a driver's request.*
- New hooks may be added to the drivers: `runtime_suspend()`, `runtime_resume()`, `runtime_idle()` in the struct dev_pm_ops structure of struct device_driver.
- API and details on power/runtime_pm:
  - pm_runtime_enable()
  - pm_runtime_get_sync()
  - pm_runtime_put()
  - pm_runtime_disable()
- Runtime PM may use resets, clocks, power...
  - It sometimes replaces direct clock lookups and handling on some old ARM platforms, like the AM335x.



Useful resources

- power/ in kernel documentation.
- Introduction to kernel power management
  Kevin Hilman (Kernel Recipes 2015)
  - https://www.youtube.com/watch?v=juJJZORgVwI
- Overview of Generic PM Domains
  Kevin Hilman (Kernel Recipes 2017)
  - https://www.youtube.com/watch?v=SctfvoskABM
- Linux Power Management Features, Their Relationships and Interactions
  Théo Lebrun (Embedded Linux Conference Europe 2024)
  - https://www.youtube.com/watch?v=_jb6U40ZCZk

Practical lab - I/O memory and ports



- Add UART devices to the board device tree
- Access I/O registers to control the device and send first characters to it.

The misc subsystem



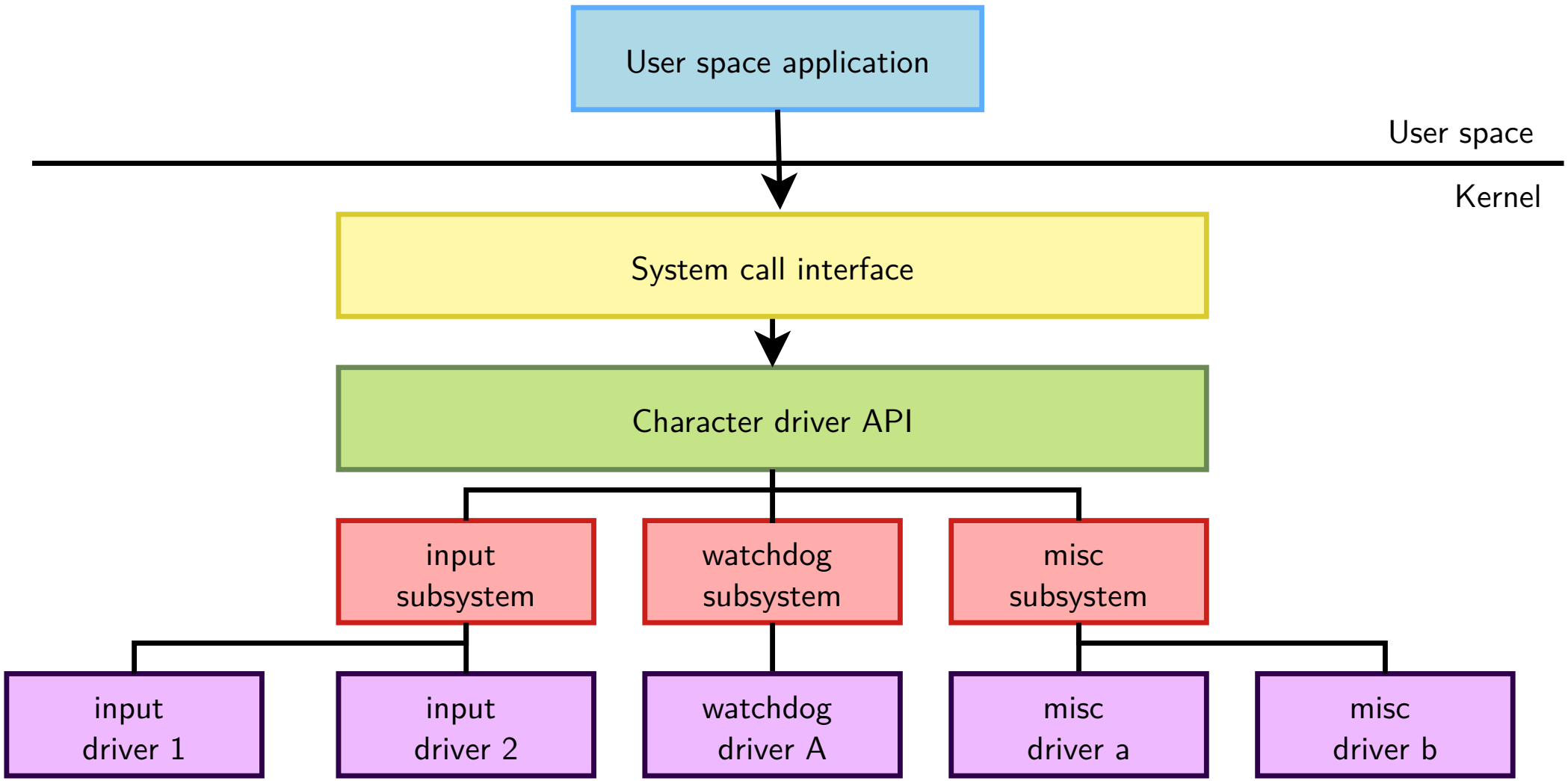
© Copyright 2004-2026, Bootlin
Creative Commons BY-SA 3.0
Corrections, suggestions, contributions and translations are welcome!



Why a *misc* subsystem?

- The kernel offers a large number of **frameworks** covering a wide range of device types: input, network, video, audio, etc.
  - These frameworks allow to factorize common functionality between drivers and offer a consistent API to user space applications.
- However, there are some devices that **really do not fit in any of the existing frameworks**.
  - Highly customized devices implemented in a FPGA, or other weird devices for which implementing a complete framework is not useful.
- The drivers for such devices could be implemented directly as raw *character drivers* (with cdev_init() and cdev_add()).
- But there is a subsystem that makes this work a little bit easier: the **misc subsystem**.
  - It is really only a **thin layer** above the *character driver* API.
  - Another advantage is that devices are integrated in the Device Model (device files appearing in *devtmpfs*, which you don't have with raw character devices).

Misc subsystem diagram





Misc subsystem API (1/2)

- The misc subsystem API mainly provides two functions, to register and unregister **a single** *misc device*:
  - `int misc_register(struct miscdevice * misc);`
  - `void misc_deregister(struct miscdevice *misc);`
- A *misc device* is described by a struct miscdevice structure:

```
struct miscdevice  {
        int minor;
        const char *name;
        const struct file_operations *fops;
        struct list_head list;
        struct device *parent;
        struct device *this_device;
        const char *nodename;
        umode_t mode;
};
```



Misc subsystem API (2/2)

The main fields to be filled in struct miscdevice are:

- `minor`, the minor number for the device, or MISC_DYNAMIC_MINOR to get a minor number automatically assigned.
- `name`, name of the device, which will be used to create the device node if *devtmpfs* is used.
- `fops`, pointer to the same struct file_operations structure that is used for raw character drivers, describing which functions implement the *read*, *write*, *ioctl*, etc. operations.
- `parent`, pointer to the `struct device` of the underlying "physical" device (platform device, I2C device, etc.)



User space API for misc devices

- *misc devices* are regular character devices
- The operations they support in user space depends on the operations the kernel driver implements:
  - The `open()` and `close()` system calls to open/close the device.
  - The `read()` and `write()` system calls to read/write to/from the device.
  - The `ioctl()` system call to call some driver-specific operations.

Practical lab - Output-only serial port driver



- Extend the driver started in the previous lab by registering it into the *misc* subsystem.
- Implement serial output functionality through the *misc* subsystem.
- Test serial output using user space applications.